



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 12
по курсу «Языки и методы программирования»
«Обработка текстовых файлов»

Студент группы ИУ9-22Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Варианты заданий для программ, которые нужно разработать в ходе выполнения данной лабораторной работы, приведены ниже. Каждая программа должна принимать через аргумент командной строки путь к каталогу, в котором располагаются подлежащие обработке файлы.

1. Найти все файлы с расширением «html» в указанном каталоге, для каждого файла определить множество гиперссылок и сохранить объединение полученных множеств в файле `links.txt` в текущем каталоге. Каждая гиперссылка в сформированном файле должна располагаться в отдельной строке. Гиперссылки должны быть отсортированы лексикографически.

Гиперссылка в HTML-файле задаётся тегом «a»: `<a href=гиперссылка>`

Работоспособность программы нужно проверить на наборе HTML-файлов, загруженных из интернета.

2. Найти все файлы с расширением «txt» в указанном каталоге, для каждого файла разбить его текст на слова, случайным образом перемешать буквы в середине каждого слова (оставить на своих местах первую и последнюю буквы) и сохранить полученный текст в файле с тем же именем, но в текущем каталоге. Работоспособность программы нужно проверить на наборе текстовых файлов, содержащих текст на английском языке.

2 Результаты

Исходный код программы представлен в листингах 1– 2.

Листинг 1: Реализация решения первой задачи

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <set>
5 #include <string>
6 #include <filesystem>
7 #include <regex>
8
9 namespace fs = std::filesystem;
10
11 std::vector<std::string> getHtmlFiles(const std::string& directory) {
```

```

12     std::vector<std::string> htmlFiles;
13     for (const auto& entry : fs::directory_iterator(directory)) {
14         if (entry.is_regular_file() && entry.path().extension() == ".
html") {
15             std::ifstream file(entry.path().string());
16             std::string line;
17             while (std::getline(file, line)) {
18                 std::smatch match;
19                 std::regex linkRegex("<a\\s+href=\"(https://[^\"]+)\"");
20                 if (std::regex_search(line, match, linkRegex)) {
21                     htmlFiles.push_back(entry.path().string());
22                     break;
23                 }
24             }
25         }
26     }
27     return htmlFiles;
28 }
29
30 std::set<std::string> getHyperlinksFromFile(const std::string& fileName)
31 {
32     std::set<std::string> hyperlinks;
33     std::ifstream file(fileName);
34     std::string line;
35     std::regex linkRegex("<a\\s+href=\"([^\"]+)\"");
36     while (std::getline(file, line)) {
37         std::smatch match;
38         while (std::regex_search(line, match, linkRegex)) {
39             hyperlinks.insert(match[1].str());
40             line = match.suffix().str();
41         }
42     }
43     return hyperlinks;
44 }
45
46 bool writeHyperlinksToFile(const std::string& fileName, const std::set<
std::string>& hyperlinks) {
47     std::ofstream file(fileName);
48
49     for (const auto& hyperlink : hyperlinks) {
50         file << hyperlink << std::endl;
51     }
52     return true;
53 }
54
55 int main(int argc, char* argv[]) {

```

```

55
56     std::string directory = argv[1];
57     std::vector<std::string> htmlFiles = getHtmlFiles(directory);
58
59
60     std::set<std::string> allHyperlinks;
61
62     for (const auto& fileName : htmlFiles) {
63         std::set<std::string> hyperlinks = getHyperlinksFromFile(
64             fileName);
65         allHyperlinks.insert(hyperlinks.begin(), hyperlinks.end());
66     }
67
68     if (!writeHyperlinksToFile("links.txt", allHyperlinks)) {
69         std::cerr << "Failed to write hyperlinks to links.txt." << std::
70             endl;
71         return 1;
72     }
73
74     std::cout << "All hyperlinks from HTML files have been saved to
75     links.txt." << std::endl;
76     return 0;
77 }

```

Результат запуска первой задачи представлен на рисунке 1.

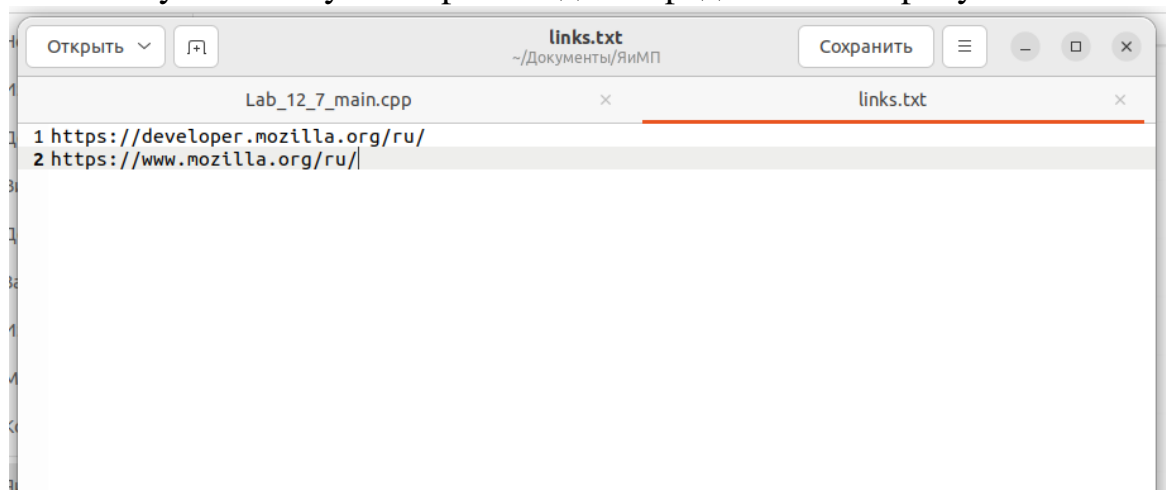


Рис. 1 — Результат 1

Листинг 2: Реализация решения второй задачи

```

1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <string>
5 #include <filesystem>

```

```

6 #include <algorithm>
7 #include <random>
8
9 namespace fs = std::filesystem;
10
11 std::vector<std::string> getWordsFromFile(const std::string& fileName) {
12     std::vector<std::string> words;
13     std::ifstream file(fileName);
14
15     std::string word;
16     while (file >> word) {
17         words.push_back(word);
18     }
19     return words;
20 }
21
22 std::string shuffleMiddleLetters(const std::string& word) {
23     if (word.length() <= 3) {
24         return word;
25     }
26     std::string middle = word.substr(1, word.length() - 2);
27     std::shuffle(middle.begin(), middle.end(), std::mt19937{std::
random_device{}}());
28     return word[0] + middle + word[word.length() - 1];
29 }
30
31 bool writeWordsToFile(const std::string& fileName, const std::vector<std
::string>& words) {
32     std::ofstream file(fileName);
33
34     for (const auto& word : words) {
35         file << word << " ";
36     }
37     return true;
38 }
39
40 int main(int argc, char* argv[]) {
41
42     std::string directory = argv[1];
43     for (const auto& entry : fs::directory_iterator(directory)) {
44         if (entry.is_regular_file() && entry.path().extension() == ".txt
") {
45             std::string fileName = entry.path().string();
46             std::vector<std::string> words = getWordsFromFile(fileName);
47             for (std::string& word : words) {
48                 word = shuffleMiddleLetters(word);

```

```

49         }
50         std::string outputFile = fs::current_path().string() + "/" +
entry.path().filename().string();
51         if (!writeWordsToFile(outputFile, words)) {
52             std::cerr << "Failed to write to file: " << outputFile
<< std::endl;
53         } else {
54             std::cout << "Processed file: " << fileName << ", saved
to: " << outputFile << std::endl;
55         }
56     }
57 }
58
59 return 0;
60 }

```

Результат запуска второй задачи представлен на рисунке 2.

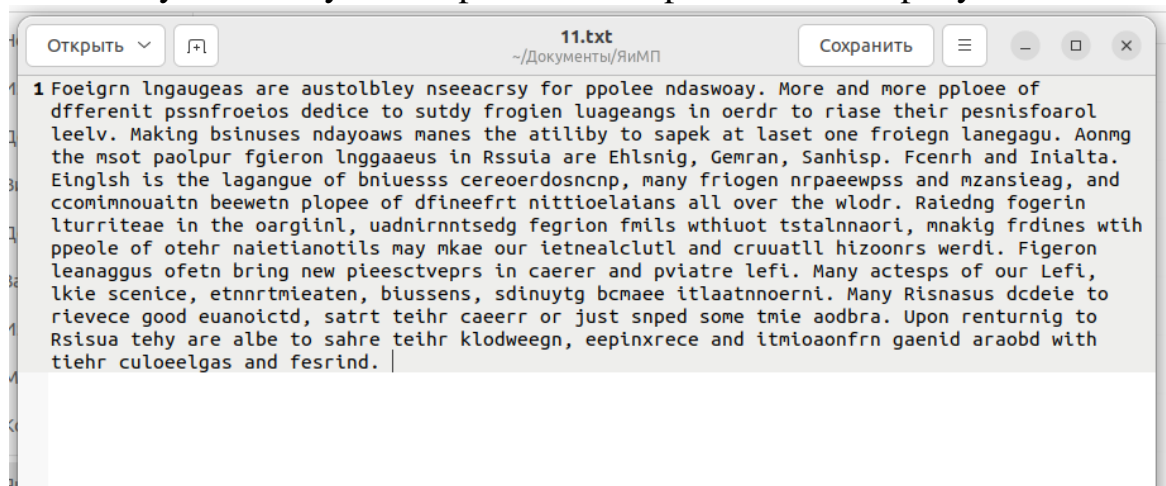


Рис. 2 — Результат 2